

ENGENHARIA DE SOFTWARE • QUALIDADE

Métricas de desenvolvimento de software

O que o mercado realmente acompanha: qualidade do código, tempo de ciclo do time e saúde do app em produção.

Métrica serve para decidir, não para julgar

Toda métrica desta aula existe para responder a uma pergunta e provocar uma ação. Se ninguém decide nada com o número, ele é decoração.

TESTE DAS TRÊS PERGUNTAS

- **Que decisão** esse número muda?
- **Quem consegue agir** sobre ele amanhã?
- O que acontece de ruim se alguém tentar **maximizá-lo**?

Lei de Goodhart

Quando uma medida vira meta, ela deixa de ser uma boa medida: as pessoas passam a otimizar o indicador, não o resultado.

```
linhas de código / dev  
commits por semana  
story points "entregues"
```

Métricas de vaidade: sobem com facilidade, não dizem nada sobre valor entregue e são triviais de inflar.

ARMADILHA

A mesma métrica muda de natureza conforme o uso. Cycle time no retrospectivo do time é diagnóstico; cycle time na avaliação individual vira alvo — e o time aprende a fraudá-lo.

Três eixos, três momentos do ciclo

As métricas mudam conforme o ponto em que você observa o software: antes do merge, durante o fluxo de entrega e depois, na mão do usuário.

EIXO 1

Código

Qualidade interna — antes do merge

Cobertura de testes

Complexidade ciclomática

Duplicação e code smells

Dívida técnica

EIXO 2

Processo

Fluxo do time — do commit à produção

Cycle time e suas fases

Tamanho de pull request

Tempo até a primeira review

Trabalho parado em espera

EIXO 3

Produção

Qualidade externa — o usuário sentindo

Crash-free users e sessions

Taxa de ANR (Android Vitals)

Cold start, jank e tamanho

Adoção de versão e retenção

Fora do escopo desta aula, mas presentes no mercado: **DORA** (frequência de deploy, lead time, MTTR, taxa de falha em mudanças) e **SPACE**, que olham a organização de entrega como um todo — assunto para uma aula própria.

Cobertura de testes

Percentual de linhas ou ramificações executadas pela suíte automatizada. Mede o que foi **exercitado** pelos testes — não o que foi verificado.

COMO SE MEDE

- Relatório por módulo, nunca só a média do app
- Separe cobertura de teste unitário e instrumentado
- Acompanhe a tendência e a cobertura do **código novo**
- `./gradlew koverHtmlReport` no pipeline

FERRAMENTAS

Kover

JaCoCo

SonarQube

Codecov

Exemplo no app

:domain	88%	regras de negócio
:data	71%	repositórios
:ui	23%	telas Compose

média	61%	

A média de **61%** esconde a informação útil: a regra de negócio está protegida e a camada de UI não. A decisão sai do módulo, não do número agregado.

ARMADILHA

Um teste que executa o código sem nenhuma asserção conta como cobertura. 100% de cobertura com zero asserts equivale a 0% de garantia — cobertura é piso mínimo, não meta a perseguir.

Complexidade ciclomática e code smells

Número de caminhos independentes de execução dentro de uma função. Quanto maior, mais casos de teste são necessários e maior a chance de bug.

COMO SE MEDE

- Detekt no CI, quebrando o build acima do limite
- Regras típicas: CyclomaticComplexMethod, LongMethod
- Limite comum: 15 por função; ajuste ao contexto do time
- Olhe a tendência do projeto, não o pico isolado

FERRAMENTAS

Detekt

ktlint

SonarQube

Android Lint

Exemplo no app

```
fun validarCadastro(f: Form): Result {  
    if (...) { if (...) { if (...) {  
        // 14 ramificações aninhadas  
    } } } } → complexidade 17
```

Extraíndo cada regra para uma função de validação: **4 funções com complexidade ≤ 4** , cada uma testável isoladamente e com nome que explica a regra.

ARMADILHA

Perseguir o número gera funções picotadas sem coesão, só para diluir a métrica. Um smell é sintoma que pede investigação, não sentença automática de refatoração.

Duplicação e dívida técnica

O SonarQube estima o **esforço em tempo** para corrigir tudo que encontrou. A razão entre esse custo e o custo de desenvolvimento é o technical debt ratio.

COMO SE MEDE

- sonar - scanner como etapa do pipeline
- Densidade de linhas duplicadas, em %
- Quality Gate aplicado ao código novo do PR
- Rating de A a E derivado do ratio

FERRAMENTAS

SonarQube

SonarCloud

CodeClimate

Exemplo no app

```
duplicação..... 4,2%  
dívida estimada... 18 dias  
debt ratio..... 3%   → A  
gate (código novo) duplicação > 3%  
                      cobertura  < 80%
```

O gate reprova o PR pelo que ele **acrescenta**, não pelo passivo do repositório inteiro — a estratégia conhecida como "clean as you code".

ARMADILHA

Mostrar a dívida do legado inteiro paralisa o time: ninguém encara "18 dias de dívida". Trave o gate no código novo e o número antigo cai sozinho, arquivo por arquivo.

Cycle time do desenvolvedor

Tempo entre o primeiro commit e o código rodando em produção. O valor cheio diz pouco; o que ensina é a **quebra em quatro fases**.

EXEMPLO — TELA DE CHECKOUT EM COMPOSE (VALORES ILUSTRATIVOS)

Coding

1º commit → PR aberto

1,5 d

Pickup

PR aberto → 1ª review

2,5 d

Review

1ª review → aprovação

1,5 d

Deploy

merge → produção

0,5 d

COMO SE MEDE

- Timestamps de commit, PR, review e deploy — via API do GitHub/GitLab
- Use a **mediana** do time, nunca a média (outliers distorcem)
- Ferramentas prontas: LinearB, Swarmia, Jellyfish, DevLake

Leitura do exemplo

Ciclo total de **6 dias**, dos quais **4 são espera** (pickup + review). Escrever código mais rápido não resolveria nada aqui: o gargalo está na fila de revisão, ou seja, é um problema de processo do time.

ARMADILHA

Usar cycle time individual em avaliação de desempenho produz PRs picotados e reviews carimbadas sem leitura. É métrica de fluxo do time, e o time é quem deve olhar para ela.

Tamanho de PR e tempo até a primeira review

As duas alavancas que mais mexem no cycle time — e as mais fáceis de um time ajustar sem mudar ferramenta nenhuma.

COMO SE MEDE

- Mediana de linhas alteradas por PR
- Horas entre abrir o PR e receber o primeiro comentário
- Exclua arquivos gerados e lockfiles da contagem
- Combine um horário fixo de review no dia

FERRAMENTAS

API do GitHub

GitLab Analytics

LinearB

DevLake

Exemplo no app

PR #412	~800 linhas	→ 3 d parado
PR #431	~150 linhas	→ 4 h até review
PR #433	~120 linhas	→ 2 h até review

Mesmo time, mesma semana. A literatura clássica de revisão aponta perda de eficácia acima de **200 a 400 linhas**: o revisor deixa de ler e passa a carimbar.

ARMADILHA

PR pequeno não pode virar cota. Quebrar uma feature em 10 PRs sem sentido próprio transfere o custo para quem revisa, que perde o contexto da mudança inteira.

Crash-free users e crash-free sessions

Percentual de usuários (ou de sessões) que não viveram nenhum crash na janela observada. **Users** mede impacto humano; **sessions** suaviza o número.

COMO SE MEDE

- Firebase Crashlytics, com alertas de velocity
- Sempre comparado **por versão** do app
- Acompanhe durante o rollout escalonado do Play
- Meta interna típica: acima de 99,5% de users

FERRAMENTAS

Crashlytics

Sentry

Android Vitals

Bugsnap

Exemplo no app

v3.1 99,6% crash-free users
v3.2 98,1% crash-free users
base: 200.000 usuários ativos

99,6% parece ótimo, mas equivale a **800 pessoas travadas**. A queda para 98,1% na v3.2 significa 3.800 usuários atingidos: motivo suficiente para pausar o rollout e investigar.

ARMADILHA

Crash-free sessions esconde o usuário que crasha toda hora: uma sessão curta e quebrada pesa o mesmo que uma sessão longa e saudável. Para decidir sobre release, olhe users.

Taxa de ANR e Android Vitals

ANR acontece quando o app não responde a uma interação — tipicamente 5 segundos de bloqueio na main thread. O Play mede isso entre os usuários ativos diários e usa o resultado no ranqueamento.

COMO SE MEDE

- Play Console → Android vitals → core vitals
- Limite geral: **0,47%** de ANR percebido
- Limite geral de crash percebido: **1,09%**
- Limite **por modelo** de aparelho: 8%

FERRAMENTAS

Android Vitals

Perfetto

StrictMode

Macrobenchmark

Exemplo no app

```
ANR geral..... 0,68%  x acima de 0,47%
Moto G (2022).... 9,1%   x acima de 8%
Pixel 7..... 0,2%   ✓
```

Causa encontrada: leitura de DataStore na main thread durante o start. A correção move a chamada para **Dispatchers.IO** — e o aparelho mais lento é justamente o que denuncia o problema.

ARMADILHA

Um número geral saudável pode esconder um modelo específico acima de 8%, e o Play reduz a visibilidade do app exatamente para os usuários daquele aparelho. Sempre abra a quebra por dispositivo.

Performance percebida: start, jank e tamanho

O usuário não percebe milissegundos médios: percebe a espera para abrir, a rolagem que trava e o download que não cabe no plano de dados.

COMO SE MEDE

- Cold start: TTID e TTFD via Macrobenchmark
- Jank: frames acima do orçamento; frozen acima de 700 ms
- Tamanho do AAB entregue, por configuração de device
- Sempre em **percentis** (p50, p90, p95) — nunca média

FERRAMENTAS

Macrobenchmark

JankStats

Baseline Profiles

Vitals

Exemplo no app

```
cold start p50.... 1,1 s  
cold start p90.... 2,4 s ← alvo  
frames congelados. 0,9% das sessões
```

Gerando um **Baseline Profile** e tirando trabalho de I/O do `Application.onCreate()`, o p90 cai para 1,5 s. A média já estava boa e não mostrava nada disso.

ARMADILHA

Medir em emulador ou em build de debug. A medição precisa ser em build de release, com R8 ativo e em aparelho de entrada — que é onde está boa parte da base de usuários no Brasil.

Adoção de versão e retenção D1 / D7

É aqui que a métrica de engenharia encontra a de produto: quanto da base já está na versão nova e se essas pessoas continuaram voltando.

COMO SE MEDE

- Adoção: % da base ativa em cada versão (Play Console)
- Retenção D1/D7: volta no dia seguinte e na semana
- Compare **coortes por versão**, não o total
- Só interprete com o rollout já concluído

FERRAMENTAS

Play Console

Firebase Analytics

GA4

Amplitude

Exemplo no app

```
v4.0  adoção 45% em 7 dias
      coorte v4.0  D7 = 18%
      coorte v3.9  D7 = 24%
```

A versão que trouxe o novo onboarding retém **6 pontos a menos** na primeira semana. O sinal não prova a causa, mas define onde investigar: o funil de primeira sessão.

ARMADILHA

Correlação não é causa. Campanha de marketing, sazonalidade e mudança de público entram na mesma janela — por isso a comparação é sempre entre coortes equivalentes.